

Fork-join parallelism

??/??/??

(drawing on materials by Dan Grossman)

Using multiple cores

- Essentially all computers now are multi-core (dual-core means 2 cores, quad-core means 4 cores, etc)
 - Each core can run all the operations needed to run a program
- Up to this point, our programs have always done a single thing at once so one can run on each core
- What if we wanted multiple cores to run a single program faster?
 - Need to split program into parts that can run on different cores

Sample problem: Summing an array

```
int simpleSum(int[] array, int low, int hi) {  
    int sum = 0;  
    for(int i=low; i < hi; i++)  
        sum = sum + array[i];  
    return sum;  
}
```

Splitting the sum into two parts

```
int sum(int[] array, int low, int hi) {  
    int left = simpleSum(array, low, (hi+low)/2);  
    int right = simpleSum(array, (hi+low)/2, hi);  
    return left + right;  
}
```

New conceptual keywords

(not real Java)

```
int sum(int[] array, int low, int hi) {
```

Allows child call (rest of line) to run in parallel with its parent

```
    int left = fork simpleSum(array, low, (hi+low)/2);
```

```
    int right = simpleSum(array, (hi+low)/2, hi);
```

```
    join;
```

Wait for **all** children to complete

```
    return left + right;
```

```
}
```

New conceptual keywords

(not real Java)

```
int sum(int[] array, int low, int hi) {
```

```
    int left = fork simpleSum(array, low, (hi+low)/2);
```

```
    int right = simpleSum(array, (hi+low)/2, hi);
```

```
    join;
```

```
    return left + right;
```

```
}
```

Allows child call (rest of line) to run in parallel with its parent

Wait for **all** children to complete

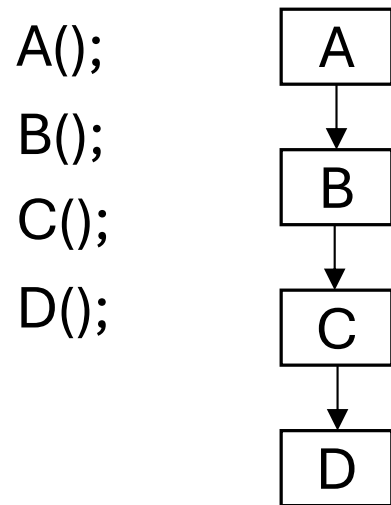
Hopefully, this

becomes these



time →

fork and join in pictures



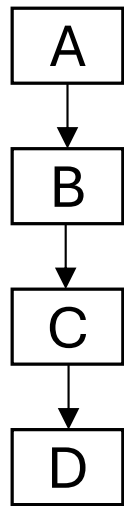
Each line must run in
the given order

Boxes are *strands* (chunk of
code we want to run without a
fork or join)

Arrows show *precedence
constraints*, relationship where
one strand must run before
another

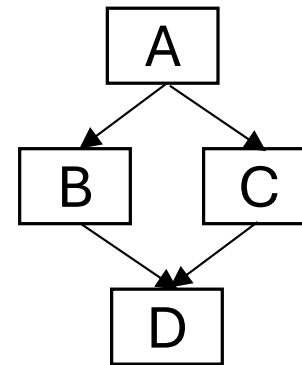
fork and join in pictures

A();
B();
C();
D();



Each line must run in
the given order

A();
fork B();
C();
join;
D();



Both B and C must run after A
and before D, but their
relationship is undefined

Which of the following could be printed by the code below?

```
fork System.out.print(1);  
System.out.print(2);  
fork System.out.print(3);  
join;  
System.out.print(4);
```

I. 3124

II. 2134

III. 1243

A. Only I.

B. Only II.

C. Only I. and II.

D. Only II. and III.

E. None of the above

Which of the following could be printed by the code below?

```
fork System.out.print(1);  
System.out.print(2);  
fork System.out.print(3);  
join;  
System.out.print(4);
```

I. 3124

II. 2134

III. 1243

A. Only I.

B. Only II.

C. Only I. and II.

D. Only II. and III.

E. None of the above

Which of the following could be printed by the code below?

```
System.out.print(1);  
fork System.out.print(2);  
System.out.print(3);  
System.out.print(4);  
join;
```

I. 1234

II. 2134

III. 1324

A. Only I.

B. Only II.

C. Only I. and II.

D. Only II. and III.

E. None of the above

Which of the following could be printed by the code below?

```
System.out.print(1);  
fork System.out.print(2);  
System.out.print(3);  
System.out.print(4);  
join;
```

I. 1234

II. 2134

III. 1324

A. Only I.

B. Only II.

C. Only I. and II.

D. Only II. and III.

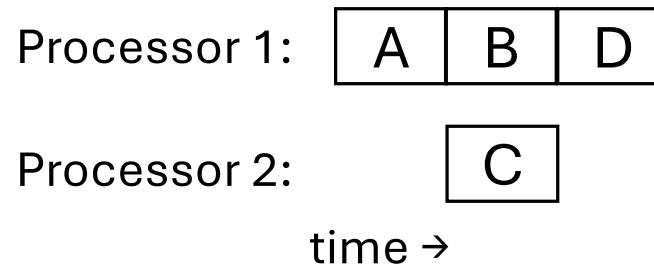
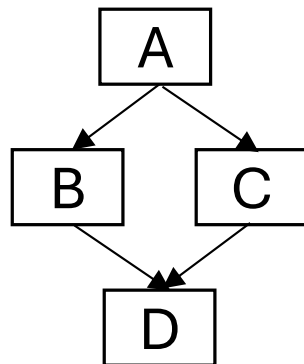
E. None of the above (I. and III.)

Running a precedence graph

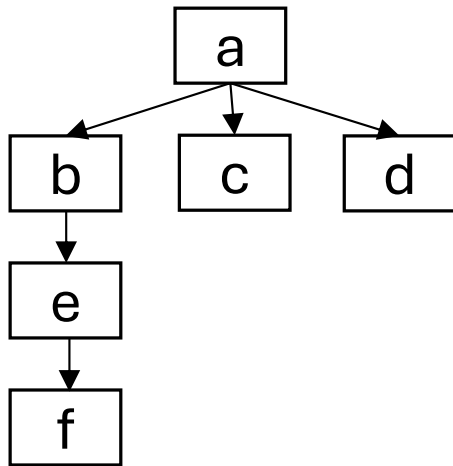
- Have a pool of cores/processors/threads (“processors”) that are available to run strands
- Each time unit, strands w/o unfinished predecessors are assigned to available processors and completed
- Typically assume strands represent equal amounts of work
- Processors are never voluntarily idle

Running a precedence graph

- Have a pool of cores/processors/threads (“processors”) that are available to run strands
- Each time unit, strands w/o unfinished predecessors are assigned to available processors and completed
- Typically assume strands represent equal amounts of work
- Processors are never voluntarily idle

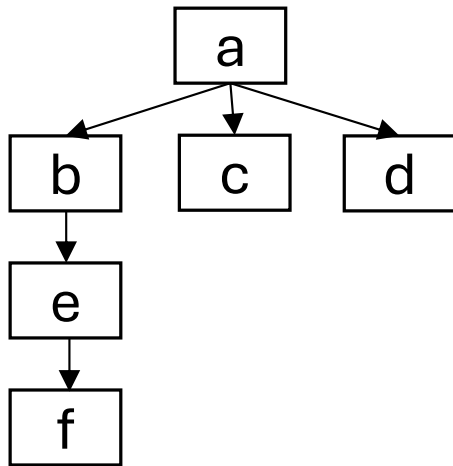


How long will this graph run on 2 processors?



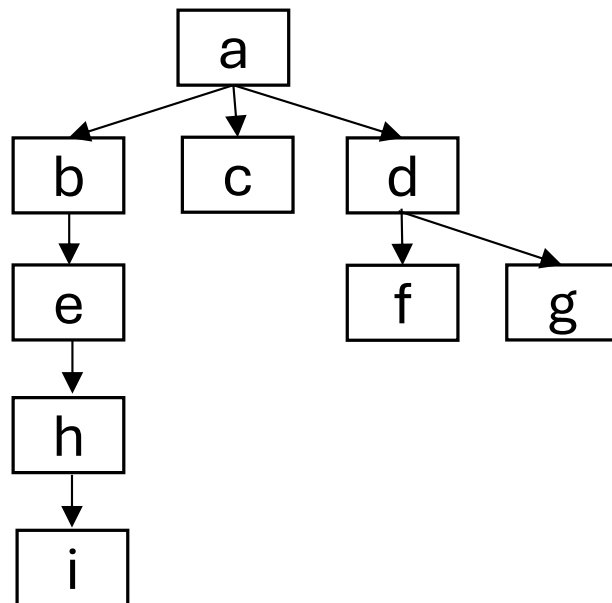
- A. Exactly 3 time units
- B. Exactly 4 time units
- C. Either 3 or 4 time units
- D. Either 4 or 5 time units
- E. None of the above

How long will this graph run on 2 processors?



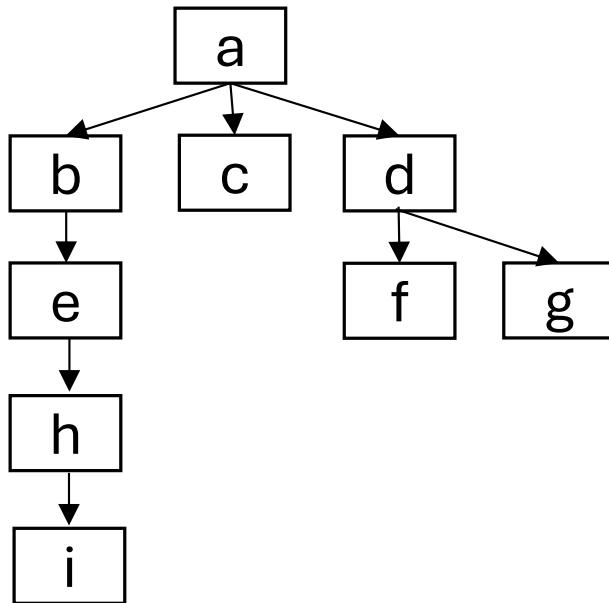
- A. Exactly 3 time units
- B. Exactly 4 time units
- C. Either 3 or 4 time units
- D. Either 4 or 5 time units
- E. None of the above

How long will this graph run on 2 processors?



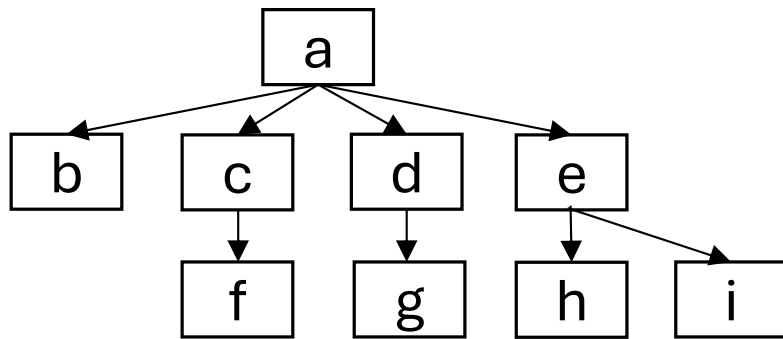
- A. Exactly 4 time units
- B. Exactly 5 time units
- C. Either 4 or 5 time units
- D. Either 5 or 6 time units
- E. None of the above

How long will this graph run on 2 processors?



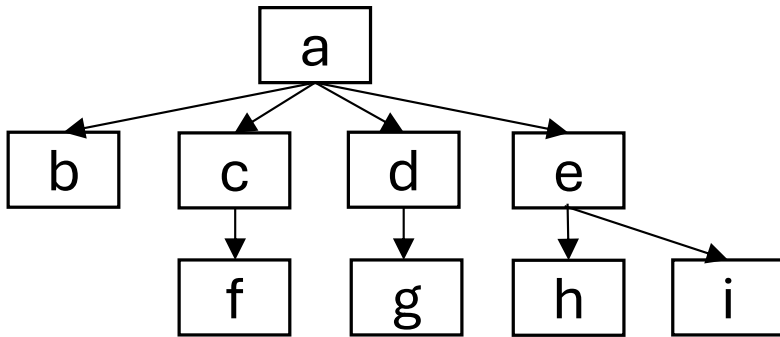
- A. Exactly 4 time units
- B. Exactly 5 time units
- C. Either 4 or 5 time units
- D. Either 5 or 6 time units
- E. None of the above
(between 5 and 7)

How long will this graph run on 2 processors?



- A. Exactly 4 time units
- B. Exactly 5 time units
- C. Either 4 or 5 time units
- D. Either 5 or 6 time units
- E. None of the above

How long will this graph run on 2 processors?



- A. Exactly 4 time units
- B. Exactly 5 time units
- C. Either 4 or 5 time units
- D. Either 5 or 6 time units
- E. None of the above

Now we're trying to optimize 2 things

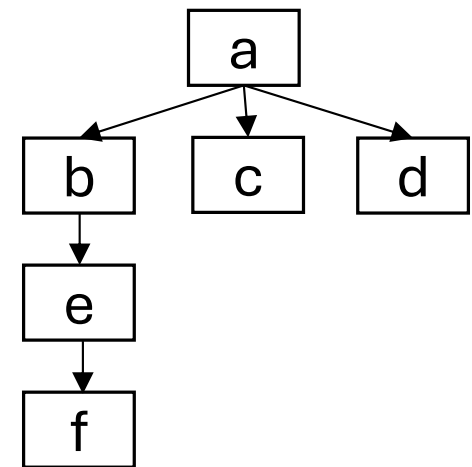
- Work = number of strands (denoted T_1)
- Span = length of longest path (denoted T_∞)

Now we're trying to optimize 2 things

- Work = number of strands (denoted T_1)
- Span = length of longest path (denoted T_∞)

What are the work and span of this graph?

- A. 3 and 4
- B. 4 and 4
- C. 6 and 3
- D. 7 and 3
- E. None of the above

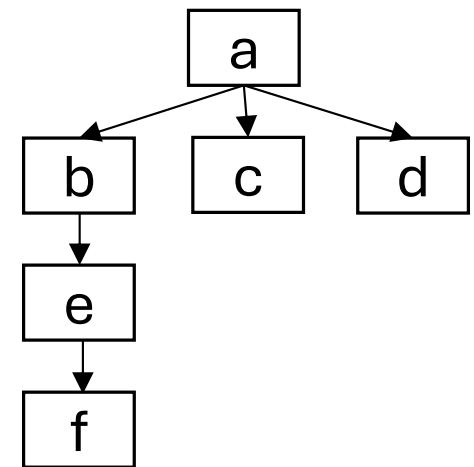


Now we're trying to optimize 2 things

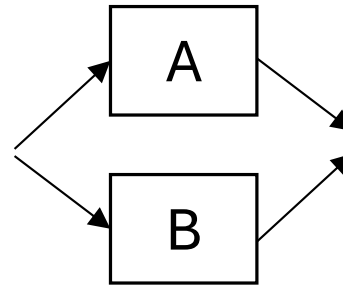
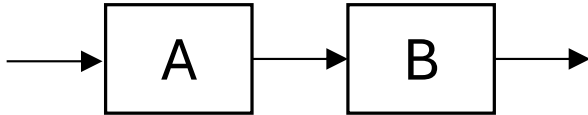
- Work = number of strands (denoted T_1)
- Span = length of longest path (denoted T_∞)

What are the work and span of this graph?

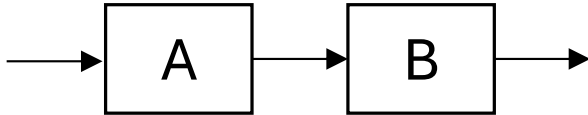
- A. 3 and 4
- B. 4 and 4
- C. 6 and 3
- D. 7 and 3
- E. None of the above (6 and 4)



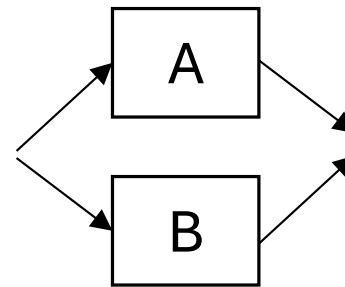
Combining subcomputations



Combining subcomputations

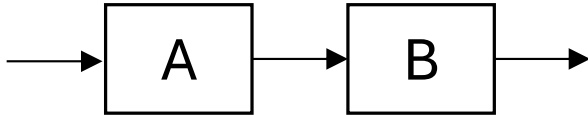


Work: $T_1(A) + T_1(B)$



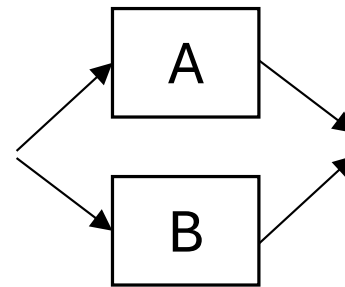
Work: $T_1(A) + T_1(B)$

Combining subcomputations



Work: $T_1(A) + T_1(B)$

Span: $T_\infty(A) + T_\infty(B)$



Work: $T_1(A) + T_1(B)$

Span: $\max\{T_\infty(A), T_\infty(B)\}$

Using different (fixed) numbers of cores

```
int sum(int[] arr, int low, int hi) {  
    int p1 = (hi+2*low)/3;  
    int p2 = (2*hi+low)/3;  
    int s1 = fork simpleSum(array, low, p1);  
    int s2 = fork simpleSum(array, p1, p2);  
    int s3 = simpleSum(array, p2, hi);  
    join;  
    return s1 + s2 + s3;  
}
```

```
int sum(int[] arr, int low, int hi) {  
    int p1 = (hi+3*low)/4;  
    int p2 = (2*hi+2*low)/4;  
    int p3 = (3*hi+low)/4;  
    int s1 = fork simpleSum(array, low, p1);  
    int s2 = fork simpleSum(array, p1, p2);  
    int s3 = fork simpleSum(array, p2, p3);  
    int s4 = simpleSum(array, p3, hi);  
    join;  
    return s1 + s2 + s3 + s4;  
}
```

Generalizing to any number of cores

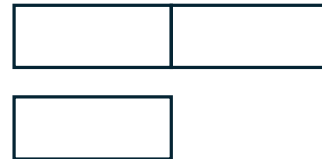
```
int sum(int[] arr, int low, int hi) {  
    int c = getNumCores();  
    int[] sums = new int[c];  
    for(int i=0; i < c; i++) {  
        int s = (i*hi + (c-i)*low) / c;  
        int e = ((i+1)*hi + (c-i-1)*low) / c;  
        sums[i] = fork simpleSum(array, s, e);  
    }  
    join;  
    return simpleSum(sums, 0, c);  
}
```

Generalizing to any number of cores

```
int sum(int[] arr, int low, int hi) {  
    int c = getNumCores();  
    int[] sums = new int[c];  
    for(int i=0; i < c; i++) {  
        int s = (i*hi + (c-i)*low) / c;  
        int e = ((i+1)*hi + (c-i-1)*low) / c;  
        sums[i] = fork simpleSum(array, s, e);  
    }  
    join;  
    return simpleSum(sums, 0, c);  
}
```

What if

... c is wrong? (the OS takes a core back)

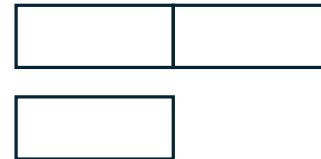


Generalizing to any number of cores

```
int sum(int[] arr, int low, int hi) {  
    int c = getNumCores();  
    int[] sums = new int[c];  
    for(int i=0; i < c; i++) {  
        int s = (i*hi + (c-i)*low) / c;  
        int e = ((i+1)*hi + (c-i-1)*low) / c;  
        sums[i] = fork simpleSum(array, s, e);  
    }  
    join;  
    return simpleSum(sums, 0, c);  
}
```

What if

... c is wrong? (the OS takes a core back)



... strands need different amounts of work?



Recursion to the rescue!

```
int sum(int[] array, int low, int hi) {  
    //base case  
    int left = fork sum(array, low, (hi+low)/2);  
    int right = sum(array, (hi+low)/2, hi);  
    join;  
    return left + right;  
}
```

Recursion to the rescue!

```
int sum(int[] array, int low, int hi) {  
    //base case  
    int left = fork sum(array, low, (hi+low)/2);  
    int right = sum(array, (hi+low)/2, hi);  
    join;  
    return left + right;  
}
```

Creates many more strands than cores

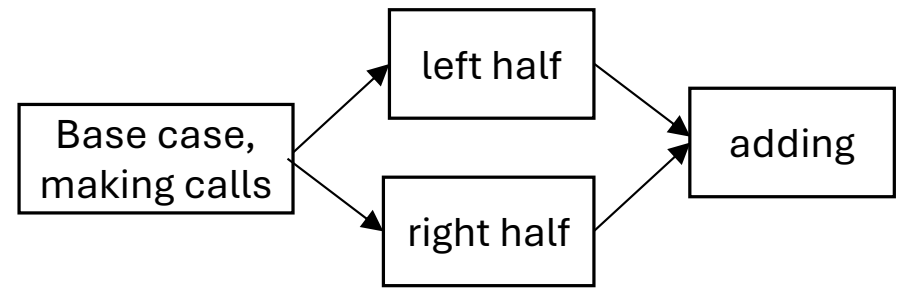
As each core finishes a strand, it gets another

Strands may have initial assignment, but any core can potentially take them (*work stealing*)

Precedence graph for this code

```
int sum(int[] array, int low, int hi) {  
    //base case
```

```
    int left = fork sum(array, low, (hi+low)/2);  
    int right = sum(array, (hi+low)/2, hi);  
    join;  
    return left + right;  
}
```



Precedence graph for this code

```
int sum(int[] array, int low, int hi) {  
    //base case
```

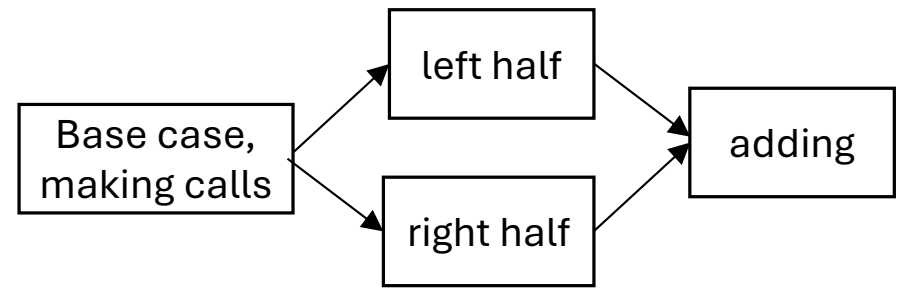
```
    int left = fork sum(array, low, (hi+low)/2);
```

```
    int right = sum(array, (hi+low)/2, hi);
```

```
    join;
```

```
    return left + right;
```

```
}
```



What is the span of this code? ($n = hi - low$)

A. $\Theta(1)$

B. $\Theta(\log n)$

C. $\Theta(n)$

D. $\Theta(n \log n)$

E. None of the above

Precedence graph for this code

```
int sum(int[] array, int low, int hi) {  
    //base case
```

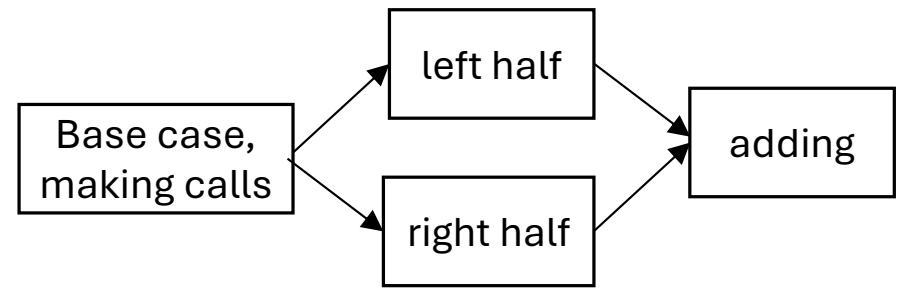
```
    int left = fork sum(array, low, (hi+low)/2);
```

```
    int right = sum(array, (hi+low)/2, hi);
```

```
    join;
```

```
    return left + right;
```

```
}
```



What is the span of this code? ($n=hi-low$)

A. $\Theta(1)$

B. $\Theta(\log n)$

C. $\Theta(n)$

D. $\Theta(n \log n)$

E. None of the above

Precedence graph for this code

```
int sum(int[] array, int low, int hi) {  
    //base case
```

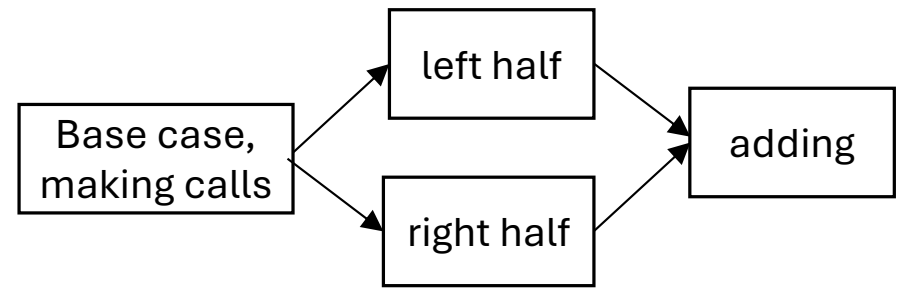
```
    int left = fork sum(array, low, (hi+low)/2);
```

```
    int right = sum(array, (hi+low)/2, hi);
```

```
    join;
```

```
    return left + right;
```

```
}
```



What is the work of this code? ($n=hi-low$)

A. $\Theta(1)$

B. $\Theta(\log n)$

C. $\Theta(n)$

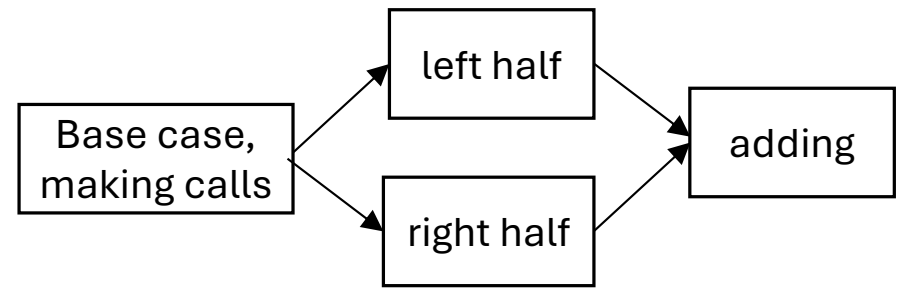
D. $\Theta(n \log n)$

E. None of the above

Applying this to a program

```
int sum(int[] array, int low, int hi) {  
    //base case
```

```
    int left = fork sum(array, low, (hi+low)/2);  
    int right = sum(array, (hi+low)/2, hi);  
    join;  
    return left + right;  
}
```



What is the work of this code? ($n=hi-low$)

A. $\Theta(1)$

B. $\Theta(\log n)$

C. $\Theta(n)$

D. $\Theta(n \log n)$

E. None of the above

What is the work and span of this code?

(n is hi-low)

```
int sum(int[] array, int low, int hi) {  
    int total = 0;  
    for(int i=low; i < hi; i++)  
        total += array[i];  
    return total;  
}
```

- A. $\Theta(\log n)$ and $\Theta(\log n)$
- B. $\Theta(\log n)$ and $\Theta(n)$
- C. $\Theta(n)$ and $\Theta(\log n)$
- D. $\Theta(n)$ and $\Theta(n)$
- E. None of the above

What is the work and span of this code?

(n is hi-low)

```
int sum(int[] array, int low, int hi) {  
    int total = 0;  
    for(int i=low; i < hi; i++)  
        total += array[i];  
    return total;  
}
```

- A. $\Theta(\log n)$ and $\Theta(\log n)$
- B. $\Theta(\log n)$ and $\Theta(n)$
- C. $\Theta(n)$ and $\Theta(\log n)$
- D. $\Theta(n)$ and $\Theta(n)$
- E. None of the above

Putting this into actual Java

```
int sum(int[] array, int low, int hi) {  
    ...  
}  
  
class Sum extends RecursiveTask<Integer> {  
    protected Integer compute() {  
        ...  
    }  
}
```

Use a class to represent a call to sum

Parameterized by the return type (must be a reference type)

Putting this into actual Java

```
int sum(int[] array, int low, int hi) {  
    ...  
}
```

Can't pass arguments into compute

Use constructor to get the information

```
class Sum extends RecursiveTask<Integer> {  
    int[] array;  int low;  int high;  
  
    public Sum(int[] array, int low, int hi) {  
        this.array = array; this.low = low; this.hi = hi;  
    }  
  
    protected Integer compute() {  
        ...  
    }  
}
```

Putting this into actual Java

```
int sum(int[] array, int low, int hi) {  
    //base case  
  
    int left = fork sum(array, low, (hi+low)/2);  
    int right = sum(array, (hi+low)/2, hi);  
    join;  
    return left + right;  
}
```

...

```
protected Integer compute() {  
    if(hi - low < THRESHOLD)  
        return array[low];  
    Sum left = new Sum(array, low, (hi+low)/2);  
    Sum right = new Sum(array, (hi+low)/2, hi);  
    left.fork();  
    return right.compute() + left.join();  
}
```

Only recurse to create
enough strands

fork for only one half
run right half recursively, but before waiting for left half

Putting this into actual Java: Starting off

```
sum = ForkJoinPool.commonPool().invoke(new Sum(array, 0, array.length));
```

Putting it all together

```
class Sum extends RecursiveTask<Integer> {  
    //attributes and constructor  
  
    protected Integer compute(){  
        if(hi - low < THRESHOLD) return simpleSum(array, low, hi);  
        Sum left = new Sum(array, low, (hi+low)/2);  
        Sum right = new Sum(array, (hi+low)/2, hi);  
        left.fork();  
        return right.compute() + left.join();  
    }  
}  
  
sum = ForkJoinPool.commonPool().invoke(new Sum(array, 0, array.length));
```

Putting it all together

```
class Sum extends RecursiveTask<Integer> {
```

```
    //attributes and constructor
```

```
    protected Integer compute(){
```

```
        if(hi - low < THRESHOLD) return simpleSum(array, low, hi);
```

```
        Sum left = new Sum(array, low, (hi+low)/2);
```

```
        Sum right = new Sum(array, (hi+low)/2, hi);
```

```
        left.fork();
```

```
        return right.compute() + left.join();
```

```
    }
```

```
}
```

What if I wanted the minimum instead of the sum?

```
sum = ForkJoinPool.commonPool().invoke(new Sum(array, 0, array.length));
```

Putting it all together

```
class Sum extends RecursiveTask<Integer> {  
    //attributes and constructor  
  
    protected Integer compute(){  
        if(hi - low < THRESHOLD) return simpleSum(array, low, hi);  
        Sum left = new Sum(array, low, (hi+low)/2);  
        Sum right = new Sum(array, (hi+low)/2, hi);  
        left.fork();  
        return right.compute() + left.join();  
    }  
}
```

What if I wanted the minimum
and its index?

```
sum = ForkJoinPool.commonPool().invoke(new Sum(array, 0, array.length));
```

Reductions

- A reduction takes a collection of values (e.g. an array) and combined them into one value
- Many useful reductions can be efficiently implemented using this framework
 - Sum, product, or combination using a boolean operator (and, or, xor)
 - Minimum or maximum (or minIndex/maxIndex)
 - Building a histogram (record of how many times each value occurs)
 - Counting number of values meeting a condition
 - IndexOf or lastIndexOf
 - Finding the longest sequence of matching values

How do we sum the values in an $n \times n$ array?

Bonus slides

- These are slides that I used after Lab 9, which asks the students to implement reductions including to find the k th smallest. Many in the class implemented the method using library sorting routines so I used these slides to show them what I intended.
- I plan to adjust the Lab 9 handout so these hopefully won't be necessary, but they're included in case they're useful. (Perhaps if you want to talk about this reduction without doing the lab.)

Finding minimum

```
int min = nums[0];  
for(int i=1; i < nums.length; i++)  
    if(nums[i] < min)  
        min = nums[i];
```

Finding second smallest

```
int first = nums[0];           //smallest so far
int second = nums[1];          //second smallest so far
if(first > second)
    swap first and second;
for(int i=2; i < nums.length; i++) {
    if(nums[i] < second) {
        second = nums[i];
        if(second < first)
            swap first and second;
    }
}
```

Finding k^{th} smallest

```
int[] smallest = new int[k];           //k smallest values so far
...
for(int i=k; i < nums.length; i++) {
    if(nums[i] < smallest[k-1]) {
        int j = k-1;
        while((j > 0) && (nums[i] < smallest[j-1])) {
            smallest[j] = smallest[j-1];
            j--;
        }
        smallest[j] = nums[i];
    }
}
```

Finding k^{th} smallest

```
int[] smallest = new int[k];           //k smallest values so far
...
for(int i=k; i < nums.length; i++) {
    if(nums[i] < smallest[k-1]) {
        int j = k-1;
        while((j > 0) && (nums[i] < smallest[j-1])) {
            smallest[j] = smallest[j-1];
            j--;
        }
        smallest[j] = nums[i];
    }
}
```

What is the running time of this?

- A. $\Theta(n)$
- B. $\Theta(n+k)$
- C. $\Theta(n \log k)$
- D. $\Theta(n^2)$
- E. None of the above

Finding k^{th} smallest

```
int[] smallest = new int[k];           //k smallest values so far
...
for(int i=k; i < nums.length; i++) {
    if(nums[i] < smallest[k-1]) {
        int j = k-1;
        while((j > 0) && (nums[i] < smallest[j-1])) {
            smallest[j] = smallest[j-1];
            j--;
        }
        smallest[j] = nums[i];
    }
}
```

What is the running time of this?

- A. $\Theta(n)$
- B. $\Theta(n+k)$
- C. $\Theta(n \log k)$
- D. $\Theta(n^2)$
- E. None of the above $\Theta(nk)$